

AI-OLMS — Maven, Git/GitHub & Docker

Exam-ready answers for Part I, Part II and Part III

Important: The answers below follow the requirements in the question paper: Java 17, WAR packaging, Maven, Tomcat, Git/GitHub, and Docker.

PART I — Maven Web Application Development [40 Marks]

1.a) Clone the repository and import it into Eclipse [4 Marks]

Git clone command:

```
git clone https://github.com/deepthisagar7/AI-OLMS.git
cd AI-OLMS
```

Eclipse import steps:

- Open Eclipse.
- File → Import → Maven → Existing Maven Projects.
- Browse to the cloned AI-OLMS folder.
- Select the detected pom.xml and click Finish.
- Wait for Maven dependencies to download and Project → Clean/Build if required.

1.b) Correct pom.xml for a Java 17 WAR application [10 Marks]

The important corrections are: use **war** packaging, configure the compiler for Java 17, and keep the project as a normal Maven web application. A suitable corrected portion is:

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    https://maven.apache.org/xsd/maven-4.0.0.xsd">

  <modelVersion>4.0.0</modelVersion>

  <groupId>com.example</groupId>
  <artifactId>ai-olms</artifactId>
  <version>1.0-SNAPSHOT</version>

  <packaging>war</packaging>

  <properties>
    <maven.compiler.release>17</maven.compiler.release>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  </properties>

  <build>
    <plugins>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-compiler-plugin</artifactId>
        <version>3.13.0</version>
        <configuration>
          <release>17</release>
        </configuration>
      </plugin>

      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-war-plugin</artifactId>
        <version>3.4.0</version>
      </plugin>
    </plugins>
  </build>
</project>
```

Why: packaging=war makes Maven produce a WAR. release=17 tells the compiler to compile for Java 17. The WAR plugin packages the web application for deployment to Tomcat.

Exam note: If the question asks only for the relevant corrected portion, you do not need to rewrite all existing dependencies; keep the project's existing valid dependencies and correct the packaging/properties/build configuration.

1.c) Maven lifecycle phases [5 Marks]

Phase	Meaning
clean	Deletes the previous build output, especially the target/ directory.
compile	Compiles the main Java source code into target/classes.
test	Compiles and runs unit tests (unless tests are skipped).
package	Packages the compiled application into its distributable format; for this project it creates the WAR.
install	Installs the generated artifact into the local Maven repository (~/.m2/repository) so other local projects can use it.

WAR is generated during the package phase: mvn package or mvn clean package.

1.d) Clean and build [3 Marks]

```
mvn clean package
```

If tests should be skipped: mvn clean package -DskipTests.

1.e) Java source/target version error [6 Marks]

First identify which Java is installed and which Java Maven is actually using. These can differ.

```
java -version
javac -version
mvn -version
```

Troubleshooting steps:

- If java -version is not Java 17, install/select a JDK 17 installation.
- Check JAVA_HOME. On Windows: echo %JAVA_HOME%. It should point to the JDK 17 directory.
- Ensure %JAVA_HOME%\bin is available on PATH, or update the system/user environment variables.
- Restart CMD/Eclipse after changing environment variables so Maven receives the new environment.
- Run mvn -version again. The Maven output must show Java version 17.
- Ensure pom.xml uses release/source/target 17. If Maven runs on Java 8, a project configured for Java 17 cannot be compiled correctly.

```
:: Windows checks
echo %JAVA_HOME%
where java
where mvn
java -version
mvn -version
```

A typical root cause is: **pom.xml requests Java 17 but Maven is running under JDK 8.** Fix JAVA_HOME/PATH or configure Eclipse's installed JRE/JDK and Maven runtime to use JDK 17.

1.f) WAR builds but fails on Tomcat [6 Marks]

1. Wrong/incompatible Java or Tomcat version. The application may require Java 17 while Tomcat is running with an older JDK. Verify with java -version on the Tomcat host and check Tomcat startup logs.

2. WAR/context-path or deployment problem. A WAR named AI-OLMS.war normally deploys under the /AI-OLMS context. Check Tomcat's webapps directory and try the correct URL. Rename to ROOT.war if the application must be served at /.

3. Application/dependency/configuration error. Missing dependencies, incompatible servlet/JSP APIs, database configuration, or invalid web.xml/application configuration can prevent deployment. Check Tomcat logs, especially deployment/SEVERE/Exception messages, and inspect the WAR contents.

```
# Useful checks
```

```
ls $CATALINA_HOME/webapps
tail -f $CATALINA_HOME/logs/catalina.out

# Windows: inspect
%CATALINA_HOME%\webapps
%CATALINA_HOME%\logs
```

1.g) Different Java versions across developers [6 Marks]

1. Identify the mismatch: run `java -version`, `javac -version`, and `mvn -version` on both machines. Compare the JDK and Maven runtime versions.

2. Require Java 17 with Maven: configure the compiler for release 17 and, for stronger enforcement, use Maven Enforcer:

```
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-enforcer-plugin</artifactId>
  <version>3.5.0</version>
  <executions>
    <execution>
      <id>enforce-java</id>
      <goals><goal>enforce</goal></goals>
      <configuration>
        <rules>
          <requireJavaVersion>
            <version>[17,18)</version>
          </requireJavaVersion>
        </rules>
      </configuration>
    </execution>
  </executions>
</plugin>
```

3. Why standardize? The same JDK avoids compiler differences, unsupported language/API errors, dependency/plugin incompatibilities, and 'works on my machine' problems. Development, CI/build servers, and deployment should use the same supported JDK major version.

PART II — Git & GitHub [40 Marks]

2.a) Initialize, commit and push the project [6 Marks]

```
git init
git status
git add .
git commit -m "Initial commit"
git remote add origin https://github.com/deepthisagar7/AI-OLMS.git
git branch -M main
git push -u origin main
```

If the remote already exists, check it with `git remote -v` and use `git remote set-url origin ...` instead of adding it again.

2.b) Correct the last commit message before push [4 Marks]

```
git commit --amend -m "Added Assignment Module"
```

Because the commit has not been pushed, amending it is safe and changes only the latest local commit.

2.c) Create and switch to feature/course-recommendation [4 Marks]

```
git switch -c feature/course-recommendation
```

Equivalent older command: `git checkout -b feature/course-recommendation`.

2.d) Recover deleted course.jsp before committing [4 Marks]

```
git restore course.jsp
```

If the file is inside a directory, provide its path, for example `git restore src/main/webapp/course.jsp`. This restores the working-tree file from the index/last committed version.

2.e) Resolve a merge conflict [8 Marks]

Scenario: feature/course-recommendation and main changed the same part of course.jsp.

```
# 1. Switch to the branch that will receive the merge
git switch feature/course-recommendation

# 2. Get latest remote information
git fetch origin

# 3. Merge main
git merge main

# 4. Identify conflicts
git status

# 5. Open the conflicted course.jsp
# Resolve sections between:
# <<<<<<< HEAD
# your changes
# =====
# main's changes
# >>>>>>> main

# 6. Stage the resolved file
git add course.jsp

# 7. Complete the merge
git commit -m "Resolve merge conflict in course.jsp"

# 8. Push the feature branch
git push -u origin feature/course-recommendation
```

If you decide the merge should be cancelled before resolving it, use `git merge --abort`.

2.f) Unstage unwanted files and create .gitignore [6 Marks]

1. Remove everything from staging without deleting files:

```
git restore --staged .
```

For one file only: `git restore --staged filename`. Older equivalent: `git reset HEAD filename`.

2. Check staged/untracked files:

```
git status
```

3. Example .gitignore:

```
# Eclipse
.classpath
.project
.settings/

# Maven
target/

# IDEs
.idea/
.vscode/

# OS
.DS_Store
Thumbs.db
```

4. Prevent future tracking: save and commit .gitignore. If a file was already tracked, .gitignore alone will not untrack it; use `git rm --cached <file>` and then commit.

```
git rm --cached <file>
git status
git add .gitignore
git commit -m "Add gitignore rules"
```

Important: `git rm --cached` removes the file from Git's index but keeps the local file on disk. `git restore --staged` . also unstages files without deleting them.

2.g) Incorrect latest implementation — pushed vs not pushed [8 Marks]

Case 1: Commit NOT pushed. If you want to rewrite/remove the latest local commit, `reset` is appropriate. For example, keep the changes in the working tree and fix them:

```
git reset --soft HEAD~1
# fix the implementation
git add .
git commit -m "Correct implementation"
```

Use `git commit --amend` instead when the implementation is correct after editing but only the latest commit needs to be replaced/updated.

Case 2: Commit ALREADY pushed. Prefer `git revert` because it creates a new commit that safely undoes the old commit without rewriting shared history:

```
git revert <commit-hash>
git push origin main
```

Avoid force-pushing rewritten history on a shared branch unless the team explicitly agrees. If history truly must be rewritten, it requires coordinated use of a force-with-lease strategy, but `git revert` is the normal safe answer for an already-pushed shared commit.

PART III — Dockerization of Maven Web Application [20 Marks]

3.a) Multi-stage Dockerfile [8 Marks]

The question specifies Java 17 and the generated WAR target/AI-OLMS.war. Use a Maven/Java 17 builder and a Tomcat/Java 17 runtime.

```
# Stage 1: Build the WAR
FROM maven:3.9-eclipse-temurin-17 AS build

WORKDIR /app

# Copy project source
COPY pom.xml .
COPY src ./src

# Build the WAR
RUN mvn clean package -DskipTests

# Stage 2: Run with Tomcat
FROM tomcat:9.0-jdk17-temurin

# Optional: remove default Tomcat applications
RUN rm -rf /usr/local/tomcat/webapps/*

# Copy WAR from the build stage
COPY --from=build /app/target/AI-OLMS.war /usr/local/tomcat/webapps/ROOT.war

EXPOSE 8080

CMD ["catalina.sh", "run"]
```

Memory trick: Builder = FROM → WORKDIR → COPY → RUN. Runtime = FROM → COPY --from → EXPOSE → CMD.

Why ROOT.war? It makes the application available at the root context /. If the WAR remains AI-OLMS.war, the usual context URL is /AI-OLMS/.

3.b) Build and run commands [4 Marks]

```
docker build -t ai-olms:latest .

docker run -d -p 7012:8080 --name ai-olms-container ai-olms:latest
```

Then access the application at <http://localhost:7012>. The mapping means **host 7012** → **container 8080**.

3.c) Four troubleshooting checks [4 Marks]

```
# 1. Is the container running?
docker ps
docker ps -a

# 2. What does Tomcat/application log show?
docker logs ai-olms-container

# 3. Is the port mapping correct?
docker port ai-olms-container

# 4. Did Tomcat receive/deploy the WAR?
docker exec -it ai-olms-container ls -l /usr/local/tomcat/webapps
```

Also verify that no other process is already using host port 7012 and that the container has not exited because of a startup/deployment error.

3.d) Container runs but browser shows 404 [4 Marks]

Possible reason 1 — context path: If the Dockerfile copies AI-OLMS.war without renaming it, Tomcat normally creates the /AI-OLMS context. Try <http://localhost:7012/AI-OLMS/> rather than the root URL.

Possible reason 2 — WAR not copied: Check the webapps directory:

```
docker exec -it ai-olms-container ls -l /usr/local/tomcat/webapps
```

You should see AI-OLMS.war and/or its exploded directory, or ROOT.war/ROOT when using the root context.

Possible reason 3 — deployment failed: Inspect Tomcat logs:

```
docker logs ai-olms-container
```

Look for deployment exceptions, SEVERE messages, missing dependencies, servlet/JSP incompatibilities, or application startup errors.

Possible reason 4 — incorrect WAR/path/configuration: Verify the WAR exists in the build stage/output and that the Docker COPY path exactly matches the filename. Linux paths and filenames are case-sensitive.

Verification summary: check `docker exec ... ls`, check `docker logs ...`, and test the correct context URL. Renaming the artifact to `ROOT.war` is the cleanest way to serve it at `/`.

Last-minute exam checklist

- Maven: `clean` → `compile` → `test` → `package` → `install`; WAR is produced at `package`.
- Java mismatch: check both `java -version` and `mvn -version`; Maven must run on JDK 17.
- Git conflict: `status` → edit conflict markers → `git add` → `git commit` → `git push`.
- Unstage safely: `git restore --staged .`; remove already-tracked ignored file with `git rm --cached`.
- Docker: build image → run with `-d` and `-p 7012:8080` → `logs/ps/port/webapps` for troubleshooting.
- WAR context: `AI-OLMS.war` → `/AI-OLMS`; `ROOT.war` → `/`.